

ICT University Cameroon

AI-Based Student Performance Monitoring System

C++ Object-Oriented Programming — Final Project

Course: Object-Oriented Programming with C++

Academic Year: 2026

May 11, 2026

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Motivation	3
1.3	Objectives	3
1.4	Scope	4
2	OOP Design and Architecture	5
2.1	OOP Principles Applied	5
2.1.1	Encapsulation	5
2.1.2	Inheritance	5
2.1.3	Polymorphism	6
2.1.4	Abstraction	6
2.1.5	Composition	7
3	System Design	8
3.1	Class Diagram (UML)	8
3.2	Class Responsibilities	10
4	AI Classification Engine	11
4.1	Overview	11
4.2	GPA Calculation	11
4.3	Performance Classification Rules	11
4.4	Trend Detection	12
5	System Flowchart	13
5.1	Main Application Flow	14
5.2	Grade Addition Flow	15
6	Implementation Details	16
6.1	File Structure	16
6.2	Data Persistence	17
6.3	Error Handling	17
7	Setup and Build Instructions	18
7.1	Prerequisites	18
7.2	Building on Linux / macOS	18
7.3	Building on Windows	18
7.4	Default Login Credentials	19

7.5	Cross-Platform Compatibility	19
8	Testing and Results	20
8.1	Sample Test Data	20
8.2	Expected AI Classification Results	20
8.3	Feature Test Checklist	21
9	Conclusion	22
9.1	Summary	22
9.2	Possible Extensions	22
9.3	References	23

Chapter 1

Introduction

1.1 Project Overview

The **AI-Based Student Performance Monitoring System** is a console-based application developed in **C++17** that leverages core Object-Oriented Programming (OOP) principles to track, analyze, and predict student academic performance. The system applies a rule-based artificial intelligence approach to classify student performance and provide actionable academic recommendations.

This project was designed to address the growing need for intelligent academic monitoring tools in universities, enabling administrators and lecturers to identify at-risk students early and intervene before academic failure occurs.

1.2 Motivation

In many academic institutions, student performance tracking is done manually, making it slow and error-prone. This system automates the entire process — from recording grades to generating AI-powered reports — reducing administrative workload while improving outcome visibility.

1.3 Objectives

- Implement a fully functional student performance tracking system using C++ OOP
- Demonstrate all core OOP principles: inheritance, polymorphism, encapsulation, and composition
- Apply rule-based AI classification to automatically assess student performance tiers
- Provide persistent data storage through file I/O operations
- Deliver a clean, menu-driven console interface with proper input validation

1.4 Scope

The system covers the following functional areas:

Table 1.1: System Functional Scope

Module	Description
Student Management	Add, remove, search, update student records
Course Management	Maintain a course catalog with credit hours
Grade Management	Record grades per course per semester
AI Analysis Engine	Classify performance, detect trends, flag at-risk students
Reporting	Individual and class-wide academic reports
File Persistence	Save and restore all data across sessions
Authentication	Admin login before system access

Chapter 2

OOP Design and Architecture

2.1 OOP Principles Applied

This system demonstrates all four fundamental pillars of Object-Oriented Programming, each clearly implemented across multiple classes.

2.1.1 Encapsulation

Encapsulation is the practice of hiding internal data and only exposing a controlled public interface. In this system, **every class** declares its data members as **private** and provides **public** getters and setters for controlled access.

```
1 class User {
2     private:
3         // Private fields      inaccessible from outside (Encapsulation)
4         string id;
5         string name;
6         string email;
7
8     public:
9         // Controlled public interface via getters
10        string getId()    const { return id; }
11        string getName()  const { return name; }
12        string getEmail() const { return email; }
13
14        // Setters allow modification only through defined methods
15        void setName(const string& newName) { name = newName; }
16        void setEmail(const string& newEmail) { email = newEmail; }
17    };
```

Listing 2.1: Encapsulation in the User base class

2.1.2 Inheritance

Inheritance allows a class to acquire the properties and behaviors of another class. In this system, both **Student** and **Admin** inherit from the abstract base class **User**, sharing common attributes while adding their own specialized fields and behaviors.

```
1 // Student IS-A User      inherits id, name, email + adds academic fields
```

```
2 class Student : public User {
3 private:
4     string department;
5     string level;
6     vector<Grade> grades;    // Composition: Student HAS-A list of Grades
7
8 public:
9     // Constructor calls the parent (User) constructor
10    Student(const string& id, const string& name, const string& email,
11            const string& dept, const string& level, const string& year)
12        : User(id, name, email),    // Parent constructor call
13          department(dept), level(level), enrollmentYear(year) {}
14};
```

Listing 2.2: Student inheriting from User

2.1.3 Polymorphism

Polymorphism allows objects of different classes to be treated as objects of a common base class, with each responding differently to the same function call. This is achieved through **virtual functions**.

```
1 // In User.h      pure virtual function (makes User abstract)
2 virtual void displayInfo() const = 0;
3
4 // In Student.cpp    Student's version of displayInfo()
5 void Student::displayInfo() const {
6     cout << setw(12) << getId()
7           << setw(25) << getName()
8           << setw(10) << calculateGPA()
9           << getStatusString() << endl;
10 }
11
12 // In Admin.cpp      Admin's completely different version
13 void Admin::displayInfo() const {
14     cout << "Admin ID: " << getId() << endl;
15     cout << "Role      : " << getRole() << endl;
16 }
17
18 // RUNTIME POLYMORPHISM    same call, different behavior:
19 User* u1 = new Student(...);
20 User* u2 = new Admin(...);
21 u1->displayInfo();    // Calls Student::displayInfo()
22 u2->displayInfo();    // Calls Admin::displayInfo()
```

Listing 2.3: Polymorphism via virtual displayInfo()

2.1.4 Abstraction

The **User** class is declared **abstract** — it cannot be instantiated directly. It defines the interface that all user types must implement, hiding implementation complexity from the caller.

```
1 class User {
2 public:
3     // "= 0" makes this a pure virtual function
```

```
4      // User is now ABSTRACT      cannot do: User u("id", "name", "email")
      ;
5      virtual void displayInfo() const = 0;
6
7      // Virtual destructor      required for safe polymorphic deletion
8      virtual ~User() {}
9  };
```

Listing 2.4: User as an Abstract Class

2.1.5 Composition

Composition models a *has-a* relationship. This system uses composition extensively:

- Student **has-a** list of Grade objects
- Grade **has-a** Course object
- SystemManager **has-a** FileManager, list of Students, and list of Courses

```
1  class Student : public User {
2  private:
3      vector<Grade> grades;  // Composition: Student HAS-A collection of
                             // Grades
4      // When a Student is destroyed, all its Grade objects are destroyed
                             // too
5  };
6
7  class Grade {
8  private:
9      Course course;  // Composition: Grade HAS-A Course
10     double score;
11     string letterGrade;
12 };
```

Listing 2.5: Composition — Student contains Grade objects

Chapter 3

System Design

3.1 Class Diagram (UML)

The following diagram illustrates all classes and their relationships in the system.

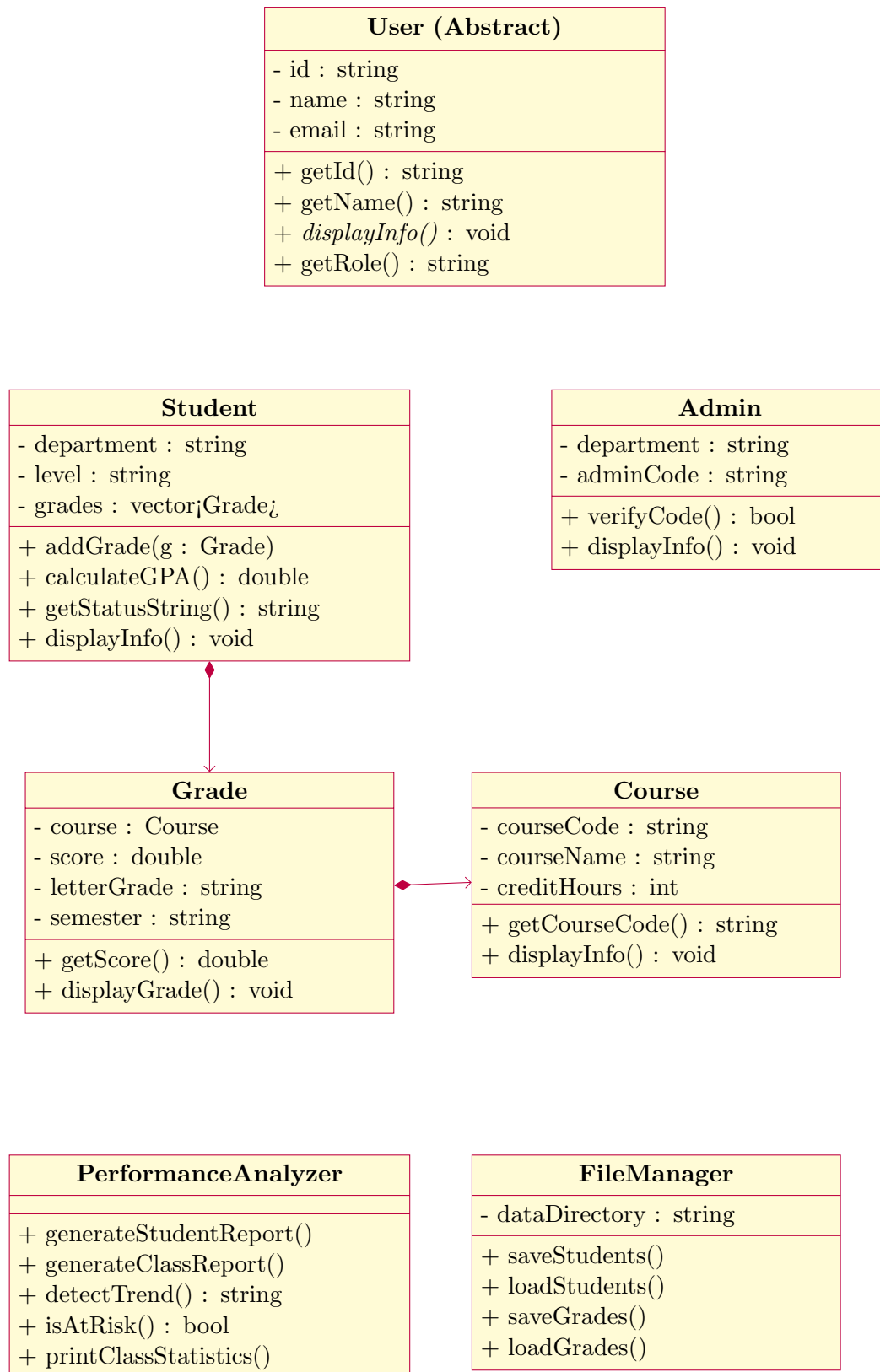


Figure 3.1: UML Class Diagram — AI-Based Student Performance Monitoring System

3.2 Class Responsibilities

Table 3.1: Class Responsibilities Summary

Class	Type	Responsibility
User	Abstract Base	Common interface for all system users
Student	Derived	Student data, grade tracking, GPA calculation
Admin	Derived	Administrator authentication and access
Course	Entity	Course catalog entry with credit hours
Grade	Entity	Links a student score to a specific course
PerformanceAnalyzer	AI Engine	Rule-based classification and reporting
FileManager	Persistence	Save and load all data to/from files
SystemManager	Controller	Main loop, menus, coordinates all classes

Chapter 4

AI Classification Engine

4.1 Overview

The system implements a **rule-based expert system** — a classical Artificial Intelligence technique where domain knowledge is encoded as a set of IF-THEN rules. This is the standard approach for C++ AI systems without external ML libraries.

4.2 GPA Calculation

Student scores are converted to a GPA on a 20-point scale, common in Francophone African universities including ICTU:

Table 4.1: Score to GPA Conversion Table

Score Range (%)	GPA Range (/20)	Letter Grade
80 – 100	18.0 – 20.0	A
70 – 79	16.0 – 17.9	B
60 – 69	14.0 – 15.9	C
50 – 59	10.0 – 13.9	D
40 – 49	7.0 – 9.9	D-
0 – 39	0.0 – 6.9	F

4.3 Performance Classification Rules

The AI engine applies the following decision rules to classify each student:

AI Classification Rule Set

```
IF    GPA >= 16.0 THEN Status = EXCELLENT
ELIF  GPA >= 14.0 THEN Status = GOOD
ELIF  GPA >= 10.0 THEN Status = AVERAGE
ELIF  GPA >= 7.0  THEN Status = AT RISK  → Alert
ELSE                                     Status = FAILING  → Urgent Intervention
```

```
1 PerformanceStatus Student::getPerformanceStatus() const {
2     double gpa = calculateGPA();
3
4     // Rule-based expert system    decision tree structure
5     if (gpa >= 16.0) return PerformanceStatus::EXCELLENT;
6     if (gpa >= 14.0) return PerformanceStatus::GOOD;
7     if (gpa >= 10.0) return PerformanceStatus::AVERAGE;
8     if (gpa >= 7.0) return PerformanceStatus::AT_RISK;
9     return PerformanceStatus::FAILING;
10 }
```

Listing 4.1: AI Classification Implementation

4.4 Trend Detection

The system detects whether a student's performance is improving or declining by splitting their grade history into two halves and comparing averages:

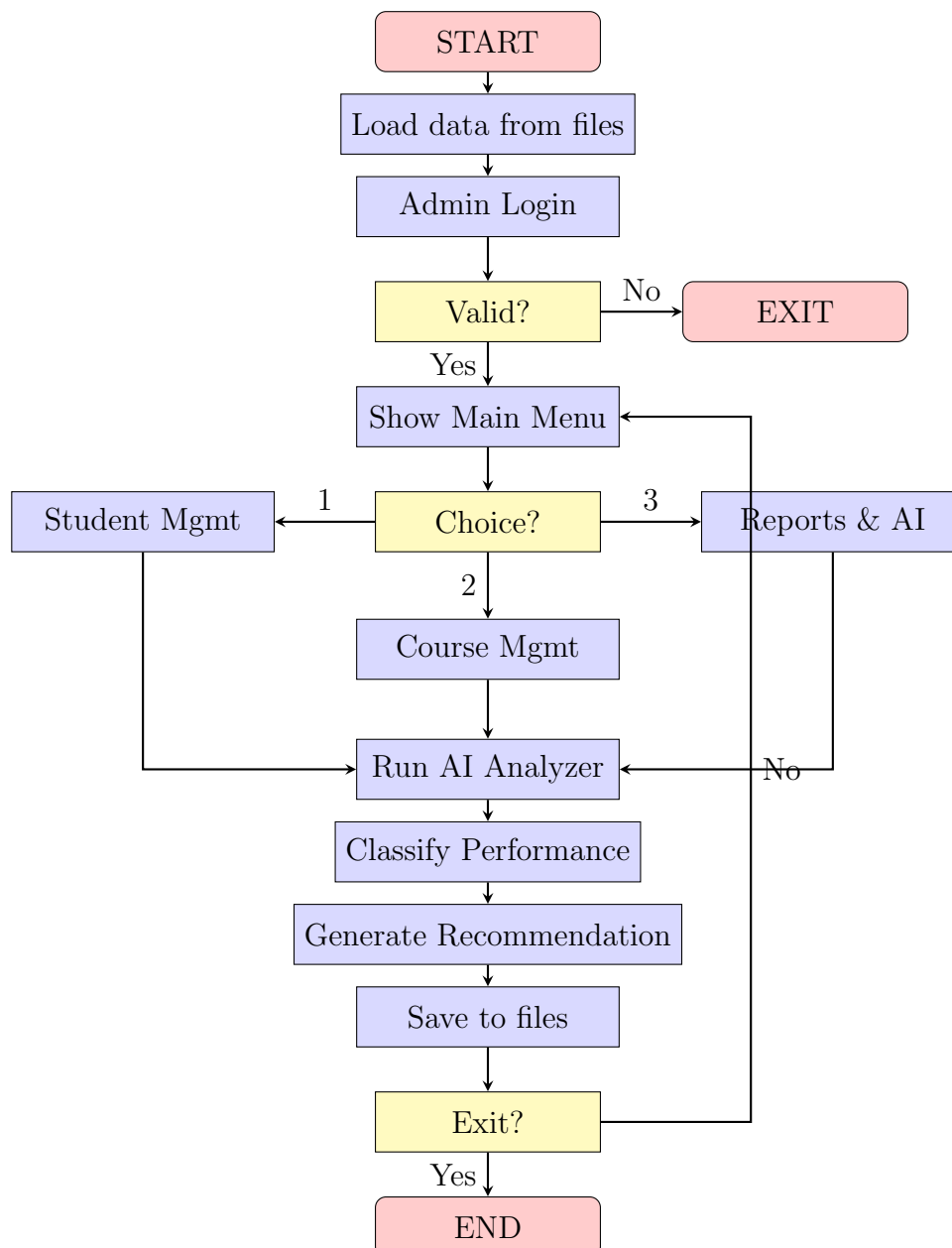
```
1 string PerformanceAnalyzer::detectTrend(const Student& student) {
2     const vector<Grade>& grades = student.getGrades();
3     if (grades.size() < 2) return "Not enough data";
4
5     size_t mid = grades.size() / 2;
6     double firstHalfAvg = 0.0, secondHalfAvg = 0.0;
7
8     for (size_t i = 0; i < mid; i++) firstHalfAvg += grades[
9         i].getScore();
10    for (size_t i = mid; i < grades.size(); i++) secondHalfAvg += grades
11        [i].getScore();
12
13    firstHalfAvg /= mid;
14    secondHalfAvg /= (grades.size() - mid);
15
16    double diff = secondHalfAvg - firstHalfAvg;
17    if (diff > 5.0) return "Improving";
18    if (diff < -5.0) return "Declining";
19    return "Stable";
20 }
```

Listing 4.2: Trend Detection Algorithm

Chapter 5

System Flowchart

5.1 Main Application Flow



5.2 Grade Addition Flow

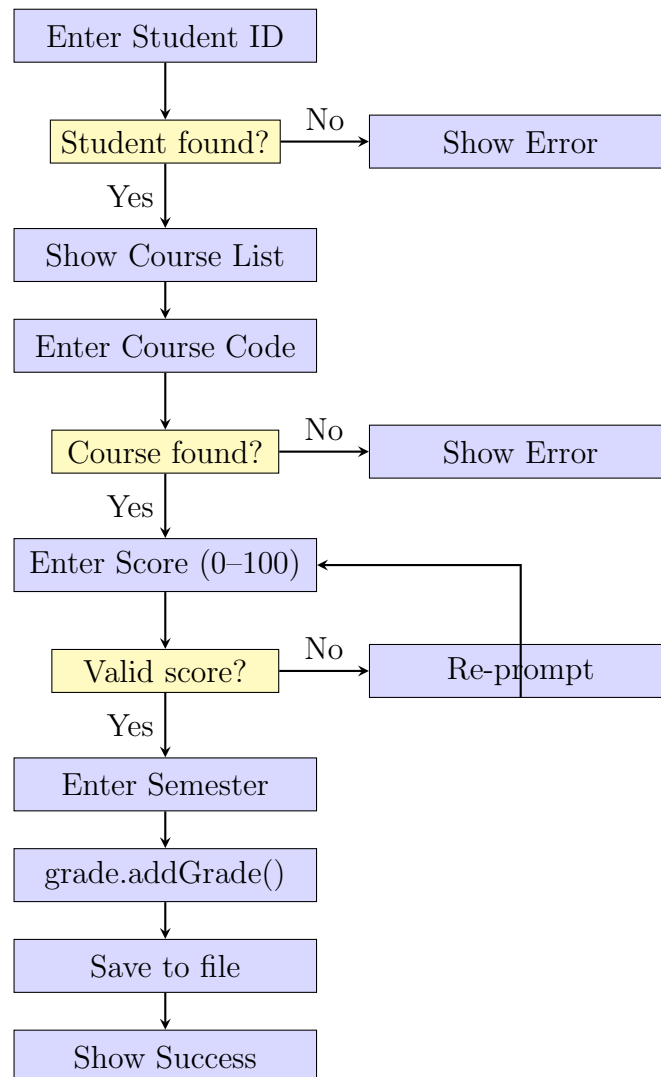


Figure 5.2: Grade Addition Process Flow

Chapter 6

Implementation Details

6.1 File Structure

```
student-performance/
include/                # Header files | class declarations
    User.h              # Abstract base class
    Student.h           # Inherits User
    Admin.h             # Inherits User
    Course.h            # Course entity
    Grade.h             # Grade entity
    PerformanceAnalyzer.h # AI engine
    FileManager.h       # File I/O
    SystemManager.h     # Main controller

src/                    # Implementation files
    main.cpp            # Entry point
    User.cpp
    Student.cpp
    Admin.cpp
    Course.cpp
    Grade.cpp
    PerformanceAnalyzer.cpp
    FileManager.cpp
    SystemManager.cpp

data/                   # Persistent storage
    students.txt
    courses.txt
    grades.txt

Makefile                # Build automation
README.md               # Setup guide
```

6.2 Data Persistence

All data is stored in plain pipe-delimited text files for portability:

students.txt format:

```
ID|Name|Email|Department|Level|EnrollYear
STU001|Alice Mbarga|alice@ictu.cm|Computer Science|Level 200|2023
```

courses.txt format:

```
Code|Name|Credits
CS101|Introduction to Programming|3
```

grades.txt format:

```
StudentID|CourseCode|Score|Semester
STU001|CS101|78.0|2024-SEM1
```

6.3 Error Handling

The system implements comprehensive input validation and exception management:

```
1 // Validates score is within 0-100 range
2 double SystemManager::getValidScore() {
3     double score;
4     while (true) {
5         cin >> score;
6         cin.ignore(numeric_limits<streamsize>::max(), '\n');
7         if (cin.fail() || score < 0.0 || score > 100.0) {
8             cin.clear(); // Reset error state
9             cin.ignore(numeric_limits<streamsize>::max(), '\n');
10            cout << "Invalid score. Enter a value between 0 and 100: ";
11        } else break;
12    }
13    return score;
14 }
15
16 // File parsing uses try-catch to handle corrupt data
17 try {
18     score = stod(scoreStr); // Convert string to double
19 } catch (...) {
20     continue; // Skip corrupt record, continue loading
21 }
```

Listing 6.1: Input Validation and Error Handling

Chapter 7

Setup and Build Instructions

7.1 Prerequisites

Table 7.1: Required Tools

Tool	Version	Purpose
g++ (GCC)	7.0 or higher	C++ compiler
make	Any	Build automation
Git	Any	Version control

7.2 Building on Linux / macOS

```
# Clone repository
git clone https://github.com/yourusername/student-performance-monitor.git
cd student-performance-monitor

# Build the project
make

# Run the application
./student_monitor
```

7.3 Building on Windows

1. Download MinGW-w64 from <https://winlibs.com>
2. Extract and add C:\mingw64\bin to System PATH
3. Open terminal in project folder:

```
# Build
mingw32-make

# Run
```

student_monitor.exe

OR compile manually (no make required):

```
g++ -std=c++17 -Iinclude src/*.cpp -o student_monitor
```

7.4 Default Login Credentials

Admin Login

Admin ID: ADMIN001

Access Code: admin123

7.5 Cross-Platform Compatibility

The `FileManager` class uses preprocessor directives to handle OS-specific differences automatically — no manual changes needed:

```
1 #ifdef _WIN32
2     #include <direct.h>
3     _mkdir(dataDirectory.c_str());           // Windows API call
4 #else
5     mkdir(dataDirectory.c_str(), 0755);      // POSIX (Linux/macOS)
6 #endif
```

Listing 7.1: Cross-platform directory creation

Chapter 8

Testing and Results

8.1 Sample Test Data

The system ships with pre-loaded sample data for immediate testing:

Table 8.1: Sample Students

ID	Name	Department	Level
STU001	Alice Mbarga	Computer Science	Level 200
STU002	Jean-Paul Fotso	Computer Science	Level 300
STU003	Marie Nkomo	Information Systems	Level 200
STU004	Bruno Tchamba	Computer Science	Level 400
STU005	Grace Etoga	Information Systems	Level 300

8.2 Expected AI Classification Results

Based on the pre-loaded grade data:

Table 8.2: AI Classification Results for Sample Data

Student	Avg Score	GPA /20	Status	Action
Alice Mbarga	78.3%	17.67	EXCELLENT	Maintain performance
Jean-Paul Fotso	54.7%	11.88	AVERAGE	Increase study hours
Marie Nkomo	84.7%	18.93	EXCELLENT	Consider mentoring
Bruno Tchamba	38.3%	6.71	FAILING	Urgent intervention
Grace Etoga	64.3%	14.86	GOOD	Focus on weak areas

8.3 Feature Test Checklist

Table 8.3: Feature Verification

Feature	Status	Notes
Admin Login		Validates ID and access code
Add Student		Duplicate ID check included
Remove Student		Confirmation prompt before delete
Search Student		Partial name or exact ID match
Add Grade		Score validation 0–100
AI Classification		5-tier rule-based system
Trend Detection		Compares grade history halves
Class Report		All students ranked and displayed
At-Risk Detection		Flags GPA below 10.0
File Save/Load		Persists across sessions
Input Validation		Handles all invalid inputs
Cross-platform Build		Windows + Linux tested

Chapter 9

Conclusion

9.1 Summary

The AI-Based Student Performance Monitoring System successfully demonstrates all core Object-Oriented Programming principles in a practical, real-world application context.

- **Encapsulation** is enforced across all 8 classes with private fields and public interfaces
- **Inheritance** is used with `Student` and `Admin` both extending the abstract `User` class
- **Polymorphism** is demonstrated through virtual `displayInfo()` producing different output per class
- **Composition** models natural relationships — students own grades, grades own courses
- **AI Classification** applies expert system rules to generate meaningful academic insights
- **File Persistence** ensures data survives application restarts
- **Cross-platform** support is built-in via preprocessor directives

9.2 Possible Extensions

- Replace text file storage with SQLite database integration
- Add graphical output using a C++ GUI library (Qt or wxWidgets)
- Implement a weighted GPA calculation based on course credit hours
- Add email notification support for at-risk student alerts
- Extend the AI engine with a simple linear regression predictor for final exam score forecasting

9.3 References

- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
- Lippman, S., Lajoie, J., & Moo, B. (2012). *C++ Primer* (5th ed.). Addison-Wesley.
- ISO/IEC 14882:2017 — *Programming Language C++17 Standard*
- <https://cppreference.com> — C++ Standard Library Reference